Week 2 Lecture

Welcome to Week 2 Lecture. In today's lecture, we will learn:

- Introduction to JS: JavaScript
- Hello world in JS: Three ways to do JS
- Variables and Constants
- Comments
- Primitive Types
- Operators
- Introduction to Methods and Functions

1: Introduction to JS - JavaScript

JavaScript, or JS, is the programming language for the web. While HTML shapes the structure of a web page, CSS shapes its styling, and JS shapes its behaviour. What happens when you hover over something or click something? What gets triggered when a user interacts with a web page? As an interaction designer, it is very important that you understand the possibilities JS offers, as this can inform your UI and UX design.

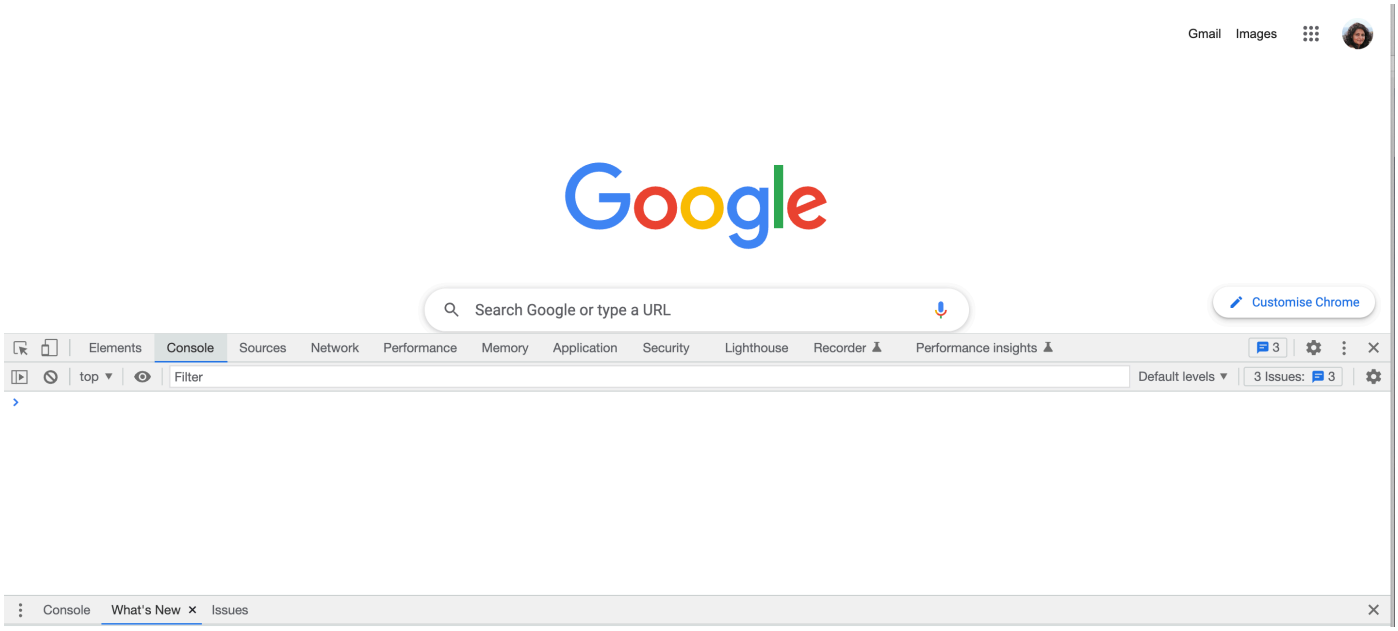
While JS was primarily developed as a programming language for the web and runs in a browser like Chrome or Firefox, libraries like Node.js enable JS to run even outside a browser. This gives you the ability to design applications, including mobile applications, outside a traditional browser environment. Every browser contains a JS engine. For example, Chrome runs on V8, and Firefox runs on SpiderMonkey. Node.js is a C++ program that wraps Google's V8 JavaScript engine and lets you write and run JavaScript outside a browser.

JavaScript has grown in popularity through the years and is now one of the leading programming languages, especially for front-end and back-end development.

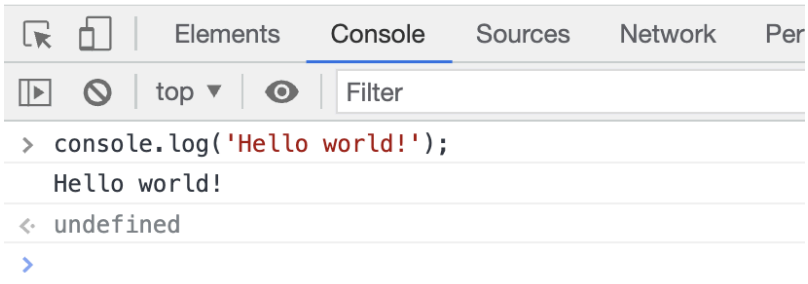
2: Hello world in JS: Three ways to do JS

2.1 Straight into the browser

If you open up an empty HTML page on Chrome and open up the Developer Tools, then the second tab is called Console. This is where you can type in JavaScript straight and see it execute interactively.



In the console, if you type in `console.log('Hello world!');`, you can see that this is printed out by the JS engine.

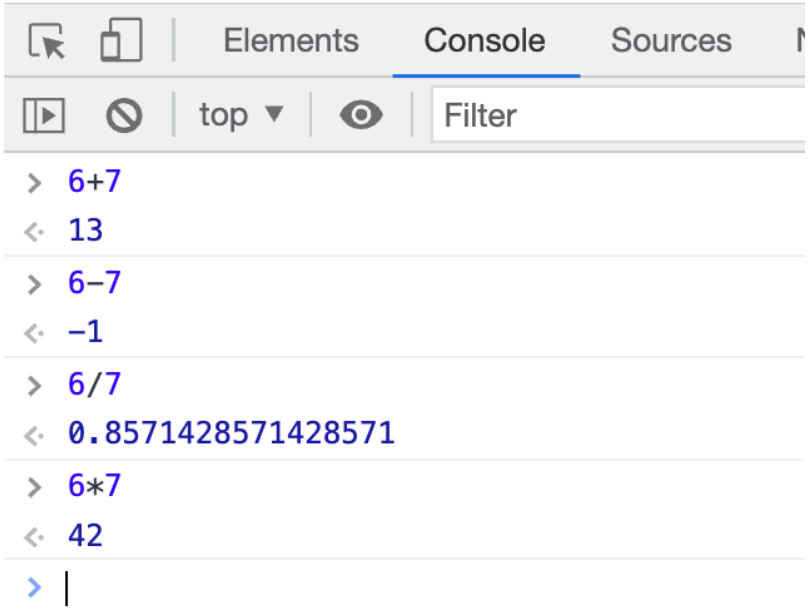


Note a few things here:

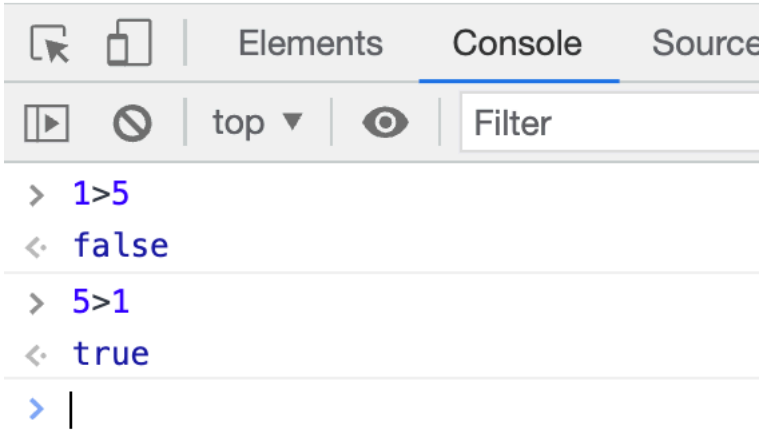
- `console.log() is an example of a function or method that takes in an input and produces an output.` Here, the purpose is to take the input string `'Hello world!'` and print it or log it to the console.
- The string `'Hello world!'` is surrounded by an opening quote and a closing quote - both single and double quotes are allowed, but be consistent and paired with whatever you choose.

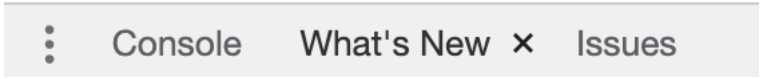
- Lastly, note that every JS statement ends with a semicolon. This signifies that the statement has ended.

You could try some other stuff here, too. For example, typing in math straight into the console computes nicely.

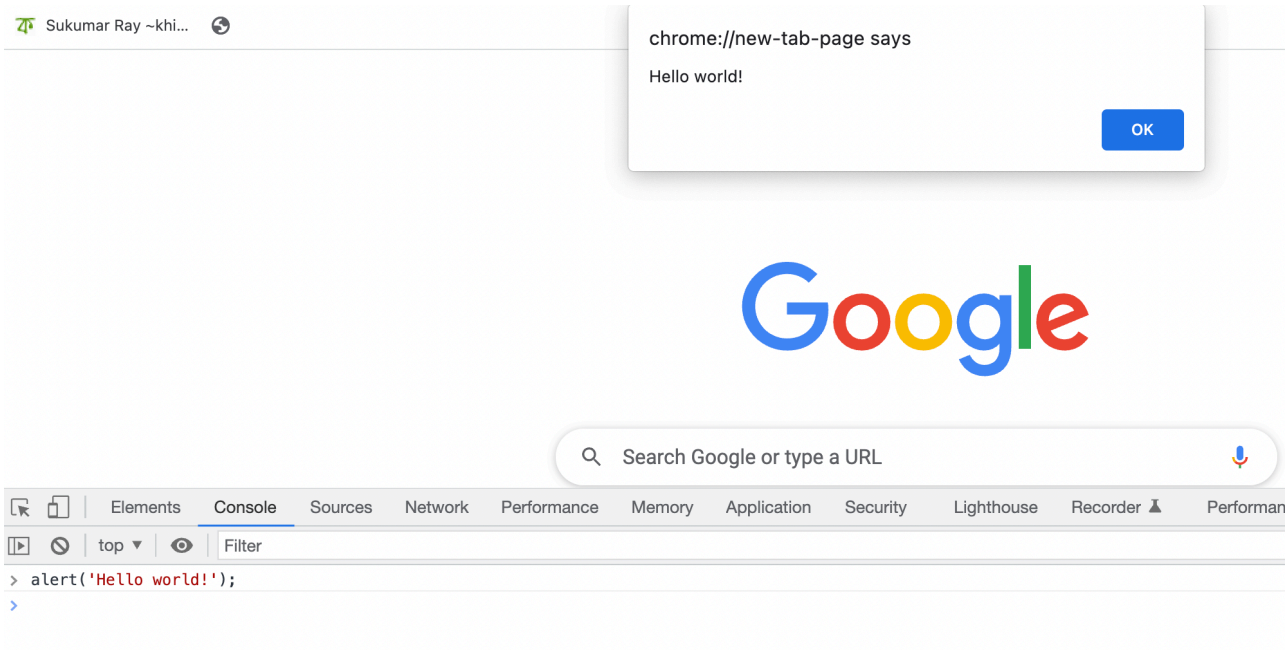


Also, Boolean operations can be typed in and evaluated. Boolean math, if you haven't come across it before, is dependent on just two values: True and False.





You can trigger an alert by using the alert function:



Important: Usually, we don't program this way, and there are many more efficient ways to write real JavaScript code (see below). However, it is useful to know that a Console is under the hood of a browser, which you can use interactively. The main use of the Console is as a **debugging tool**: when you write JavaScript programs, you can print the values of variables and the outputs from functions to the Console to test whether your program is running as you expect.

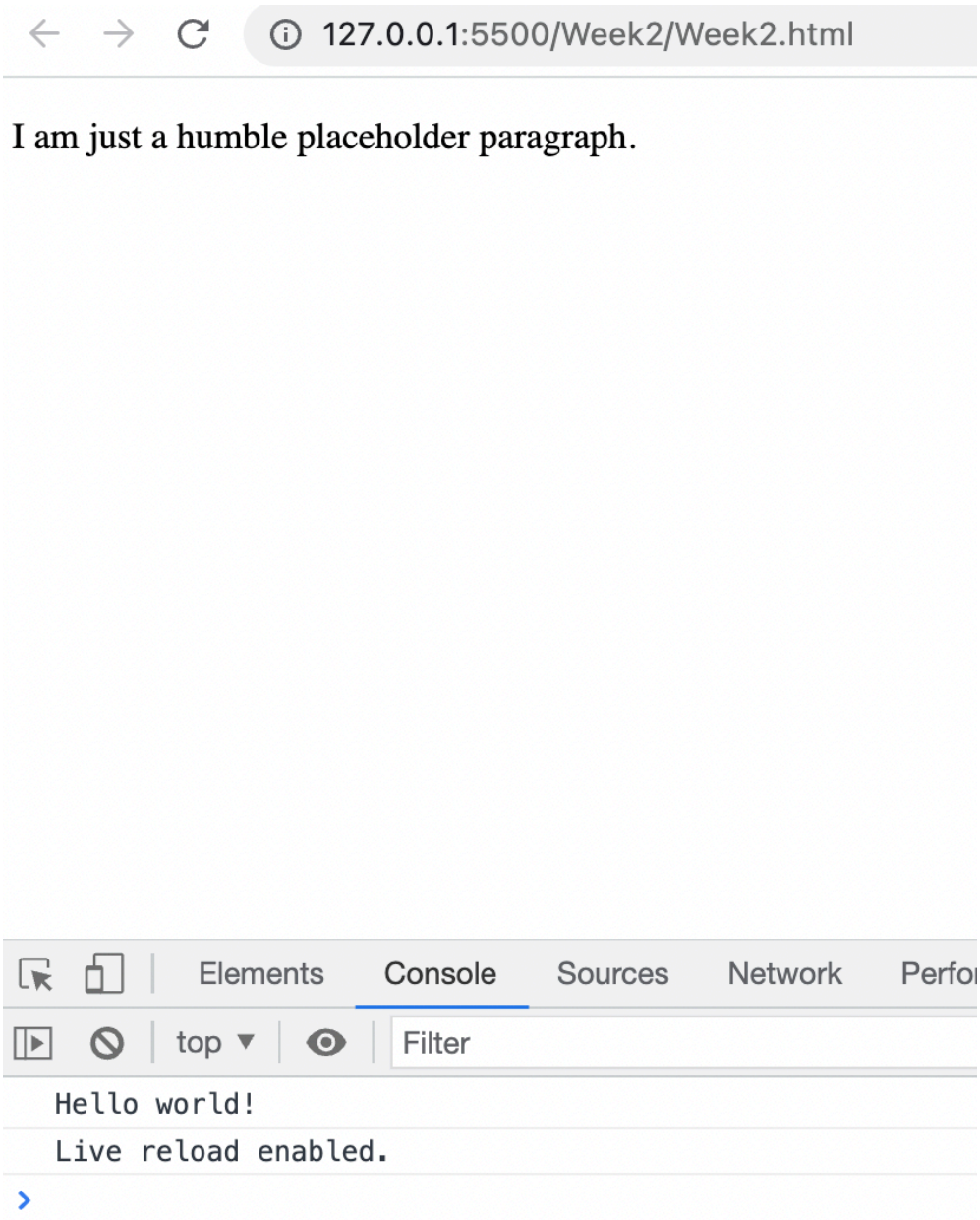
2.2 Using a `<script>` tag at the bottom of your HTML body

While it is nice to type into the browser console and see some interactive programming, this isn't a good idea for more complex, longer programs that you will eventually write. Another option is to include a <script> tag within the `<body>` tag. Typing the same code inside the tag will produce the same output when you run the page with Live Server.

Week2.html

Week2 > Week2.html > ...

```
1   <!DOCTYPE html>
2   <html>
3     <head>
4       <title>IDEA9103, Week 1</title>
5       <meta charset="UTF-8" />
6     </head>
7     <body>
8       <p>I am just a humble placeholder paragraph.</p>
9       <script>
10        console.log("Hello world!");
11      </script>
12    </body>
13  </html>
14
```



While you can put the `<script>` tag anywhere in the `<head>` or the `<body>`, it is advisable to put it at the very end. There are a couple of reasons for this. First, an HTML file is read from top to bottom. So, if you put your JS at the top and in the middle, and the instructions take time to compute, your user will sit with an empty screen, since anything written after the JS won't be executed until the JS is done. Secondly, JS is usually written to control the behaviour of HTML elements on the page - and they need to exist first before you can use the JS to modify their look or behaviour.

2.3 Using a separate `script.js` file and linking it to your HTML file

However, combining your HTML, CSS, SVG, and JS all into one HTML file is going to get cumbersome. So, the standard practice is to do what we did with the CSS - we define a separate file and name it anything, but with the .js ending format, and we link to this file from the HTML file, so that the HTML knows to look for this JS file.

Week2.html × script.js

Week2 > Week2.html > html

```
1   <!DOCTYPE html>
2   <html>
3     <head>
4       <title>IDEA9103, Week 1</title>
5       <meta charset="UTF-8" />
6     </head>
7     <body>
8       <p>I am just a humble placeholder paragraph.</p>
9       <script type="text/javascript" src="script.js"></script>
10    </body>
11  </html>
12
```

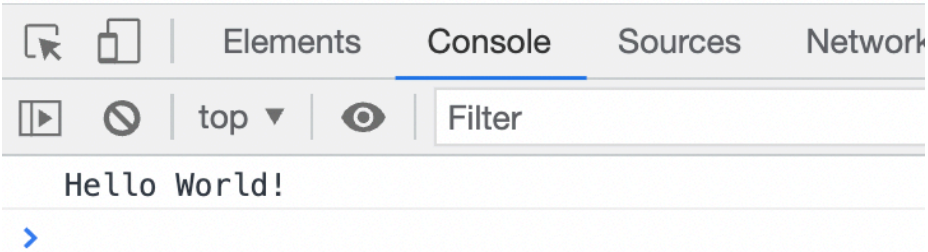
Week2.html script.js ×

Week2 > script.js

```
1   console.log('Hello World!');
```

Doing two separate files still produces the same output:

I am just a humble placeholder paragraph.



2.5: Statements, operators and terms

In JavaScript, there are several types of operators. These operators perform actions on terms. Operators and terms combine to make statements.

The most common is a binary operation. Examples of this type of operator include addition '+', multiplication '*' and assignment '='.

These operations take two arguments (terms), one to their left and one to their right, and return a value. For example, 1 + 1 returns the result of the addition operation, 2. You can try this out in the browser console if you like.

Occasionally, you may also find a unary prefix operator, such as negative '-', which can be used to express negative numbers: -123. These operators go before their argument (a term) and take only a single argument.

Some operators are post-circumfix, meaning they are placed after a term and either side of their argument, this can be seen in the parenthesis used when calling a function: console.log("my text") note that the quote operator can be conceptualised as a circumfix operator around the text input.

When curved brackets, '(' and ')',e.g., parentheses, are used to express the order of operations in a statement, they are an example of a circumfix operation: (6 – 1) / 2

3: Variables

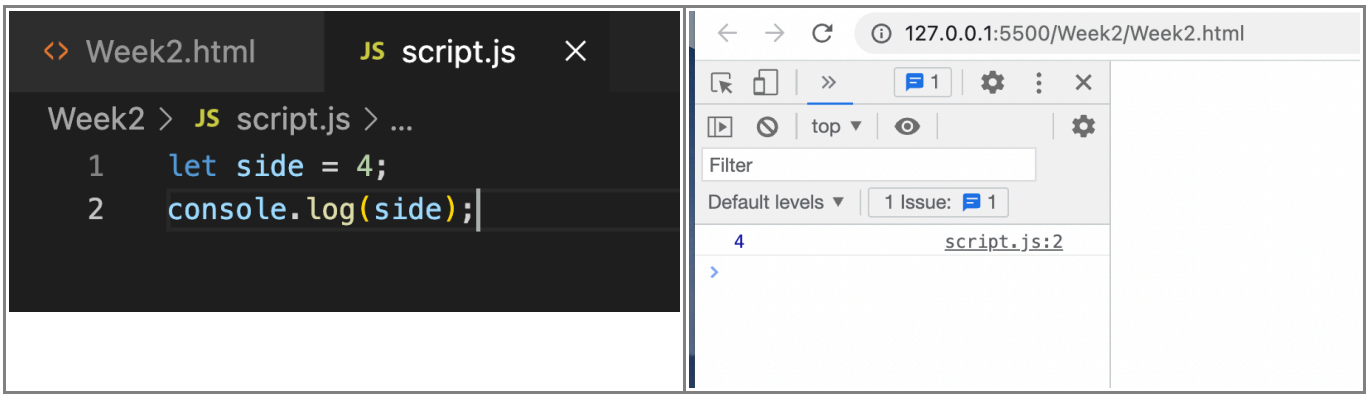
In programming, VARIABLES are used to store data in a temporary way in the memory. Usually we need to call and reuse this variable over and over again, so it is useful to name it. For example, here is the formula for computing the area of a square:

$$Area = side^2$$

Now in separate runs of a program, you can generate many squares of different areas, with different side lengths, simply by changing (or varying) the values stores in the `side` variable.

Variables are declared with the keyword `let`. In every programming language, we have a few reserved keywords - these words are used for the specific meaning they convey to the programming language. In this case, the word `let` is used to tell JS that we are declaring a variable. By default, if we now check in the browser, the variable is `undefined`, because we have not yet assigned it a value. So, its like an empty box with a name `side`, but as yet doesn't contain a value.

Now let's assign it a value, and we can check that the value is printed.



3.1 Variable Naming Rules

We should follow some simple rules when naming our variables:

- Use meaningful names. For example, I could have used `s` or `x` or `anything` as a valid variable name, but someone else reading my code (or indeed me reading my own code a few months down the line) would be confused. Use names that make sense in the context of the design you are coding, and which describe the data you're going to store in this variable.
- Do not use a reserved keyword. If you write `let let = 4;`, for example, the program will throw an error.
- Variable names cannot start with a number. So, for coding the area of a rectangle, it's ok to have variables `side1` and `side2`, but not `1side` or `2side`.
- Variable names cannot have a space or a hyphen. Instead, the standard practice is to use camelCase: like `sideOne` and `sideTwo`. Remember also that variable names are case sensitive - so `sideone` and `sidetwo` will be interpreted as two new variables and not the ones above.
- You can either declare variables on one line, separated by commas:
  - `let sideOne, sideTwo;`

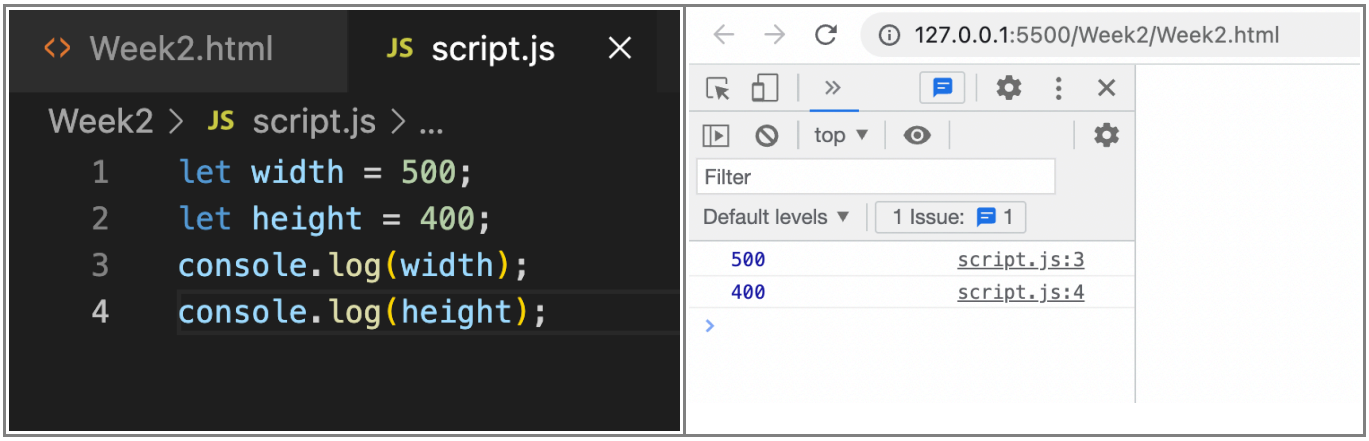
or use two separate commands on two separate lines:

  - `let sideOne;`
  - `let sideTwo;`

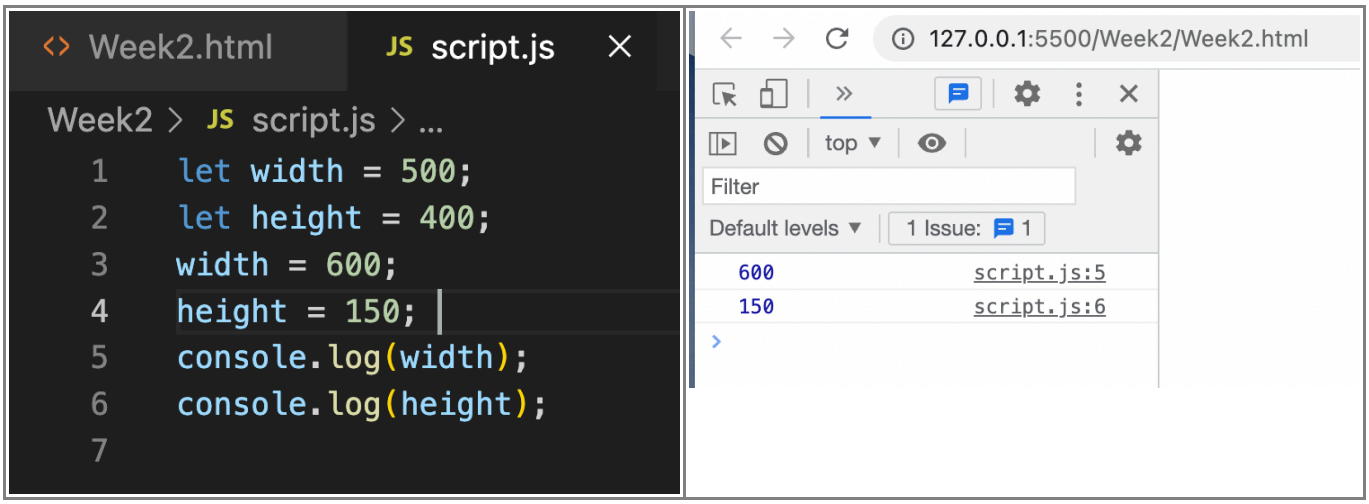
I personally prefer each variable declared on its own separate line - makes for cleaner code. Whichever way you choose, be consistent.

4: Constants

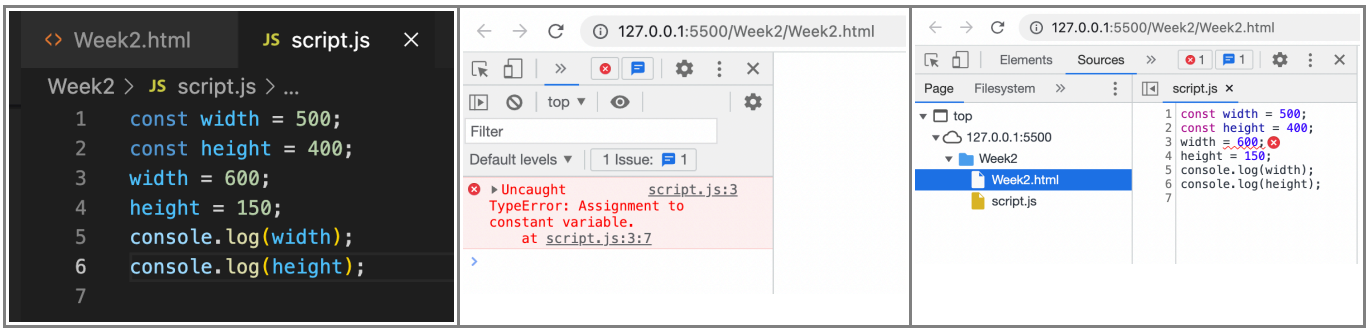
Sometimes we need to assign values to variables that are technically not supposed to "vary". For example, setting the size, width and height, of your SVG/drawing canvas. In such a case, we might need to define two variables, say `width` and `height`.



Using let, we can change these value assignments, and no errors will be thrown. You can see that the old values have been over-written by new values.



However, we don't want this behaviour to occur, since in many applications the canvas size on which we want to draw is fixed at the beginning of the project and remains fixed to the very end. In such cases, when we do not want the value to change, we use the keyword `const` instead of `let`. `const` signifies that we are declaring a CONSTANT. If you try to re-assign a value, the code will throw an error.



Therefore, a good rule of thumb is this: use const in general, and only use let when you're absolutely sure the value assigned to the variable will change during program execution.

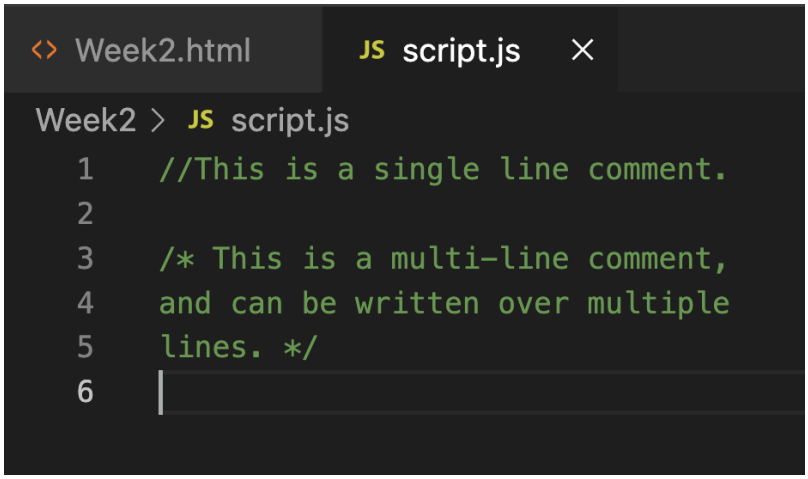
There was another keyword, var, used to declare variables in older versions of JS (pre-ES6 or pre-2015). However, as we will learn later about **scoping**, we will see why we should stick with

const and let, though some older code you find on the internet uses `var`.

5: Comments

Now that we are warming up towards writing some actual code, we should talk about commenting our code. A good programming practice is to liberally put in notes and comments for our (older) self, and for others reading your code. The comments usually are not about how the code works - that we can see just by reading the code. Instead, the comments focus on your aim and intention as a designer, and why you are doing what you're doing. For example, a comment like - "`I am declaring a variable called side`" is not as useful as "`side represents the side length of a square, which will be one of my main drawing primitives/blocks`".

The important thing to remember is that comments are non-executable statements - they are only there for our benefit to understand the code, and they are not executed when we run the program. You can write single-line or multi-line comments:



6: Primitive Types

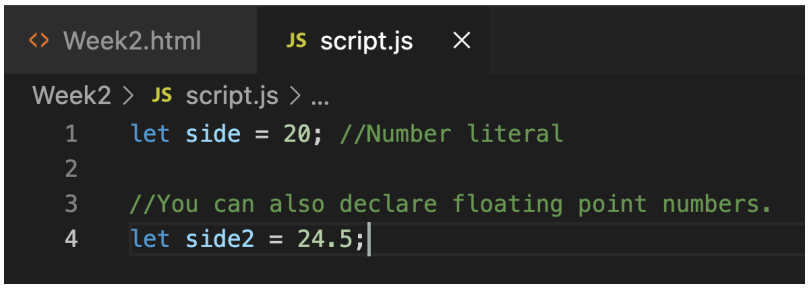
So, now that we know how to declare variables, what kind of values can we put in them? In JavaScript, we have two main categories of Data Types:

- Primitive Types
- Reference Types

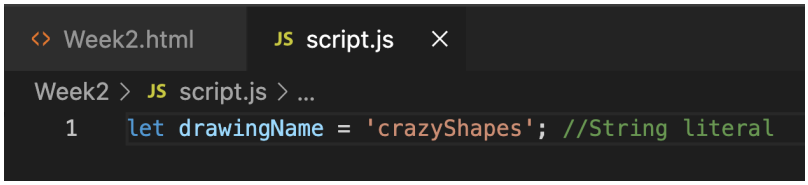
In this lecture, we are going to focus on Primitive Types, and learn about Reference Types later in the coming weeks.

There are 5 primitive types:

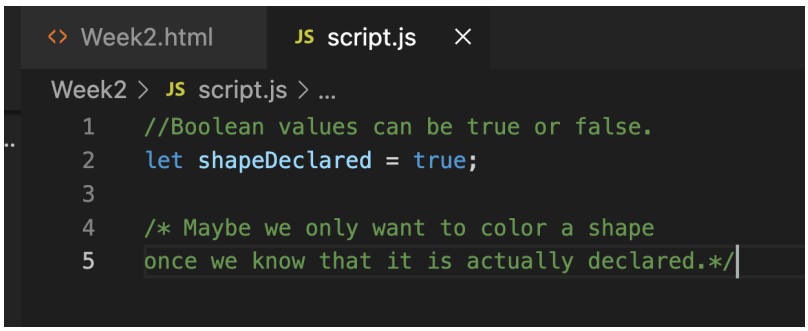
- Number



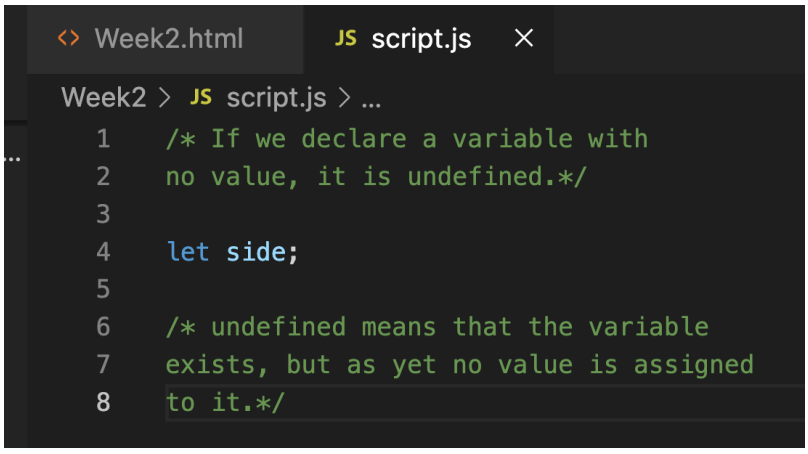
- String



- Boolean



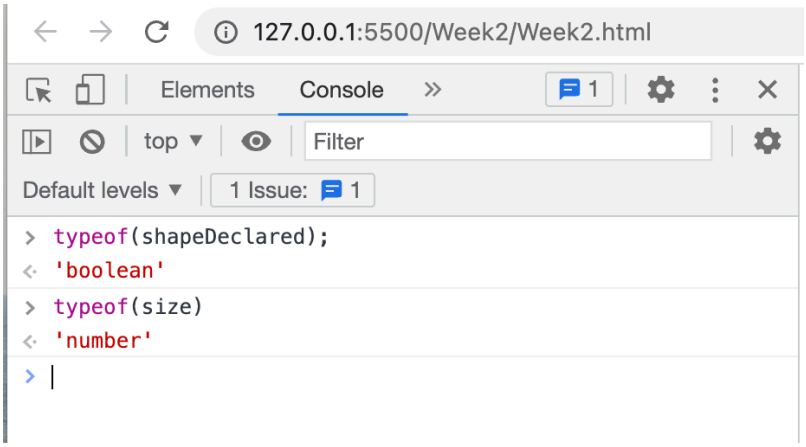
- undefined



- null

```
Week2.html   script.js ●
Week2 > script.js > ...
1   /* We can also explicitly set the value
2   to a null value.*/
3
4   let side = null;
5
6   /* This is usually used when we want to
7   erase a previous value, because the program
8   requires it.*/
9
10  /*For example, you may want to set the side
11  variable to null everytime you start off a
12  new drawing sequence within the same program.*/
13
14  /*Note that a null value is different from
15  an empty, or undefined value.*/
```

Finally, a very useful way to check the type of a variable is to use the typeof() method. We will be learning about functions and methods next, so this is like a sneak peek. Basically, you can **CALL** methods on objects to check their states. I think of a function, or method, as a machine that eats some inputs, then processes those inputs in some way, and then produces an output. So, if I put in `typeof(shapeDeclared);` right into the console, it takes in the variable `shapeDeclared`, ascertains what type of variable it is, and then outputs to me that it is Boolean. You can try the others and see what types are returned.



Pay special attention to this typeof method - in the tutorial, we will be learning to use three new, useful methods to do something interesting!

7: Operators

We have a full set of "alphabets" or primitive and reference types to work with. We can now turn our attention to how we can construct logical "sentences" or expressions, which allow us to actually do some interesting stuff. There are primarily 5 types of operators in JavaScript (some of these you've already seen and worked with before):

- Arithmetic
- Assignment
- Bitwise

Let's look at each of these in detail.

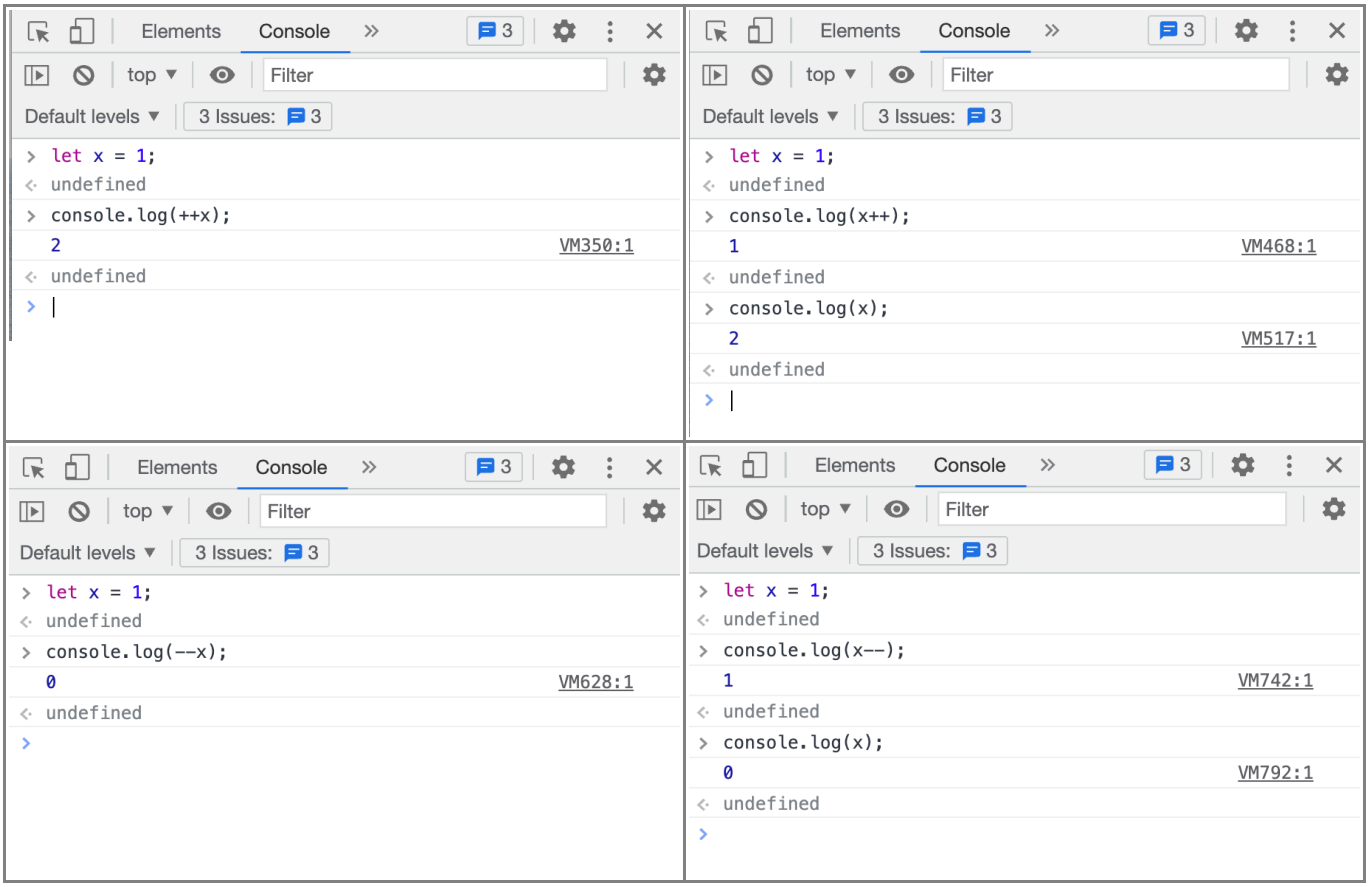
Arithmetic Operators

```
let x = 20;

let y = 3;

//Everything inside the brackets are examples of "expressions".

console.log(x+y); //Addition

console.log(x-y); //Subtraction

console.log(x*y); //Multiplication

console.log(x/y); //Division

console.log(x%y); //Mod or remainder

console.log(x**y); //Exponentiation
```

Running all the above gives us the expected outputs:

In addition, we have two other very useful arithmetic operators: Increment (++) and Decrement (--), which increase and decrease the value of the variable by 1. This is going to be very useful later, for example, when we learn about loops. Note carefully that if the ++ or the -- precedes the

variable, the incremented value is printed, whereas if the ++ or -- is typed after the variable, the old value is printed, and then the value is increased or decreased.



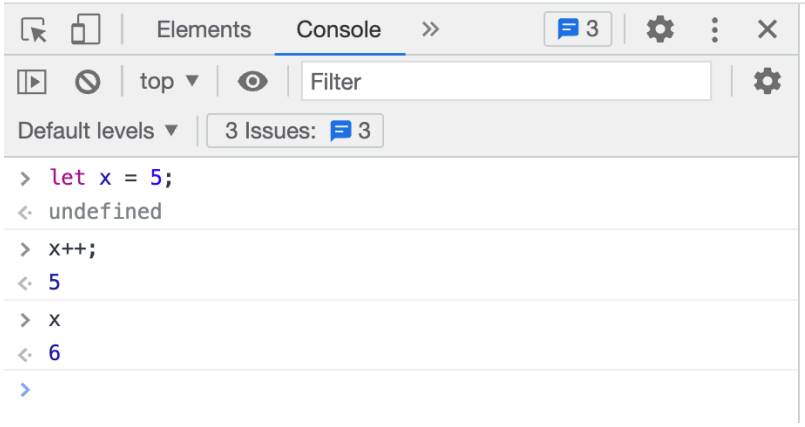
Assignment Operators

Next, we have assignment operators, which assign values to variables. You have already seen some examples:

```
let x = 5;
```

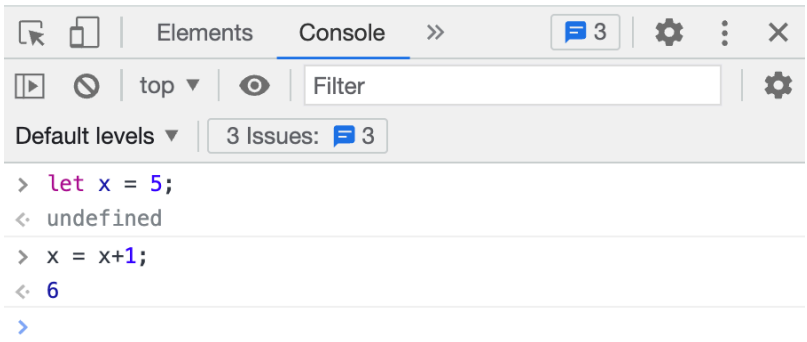
We have just seen that if we want to increase the value of x by 1, we can write:

```
x++;
```



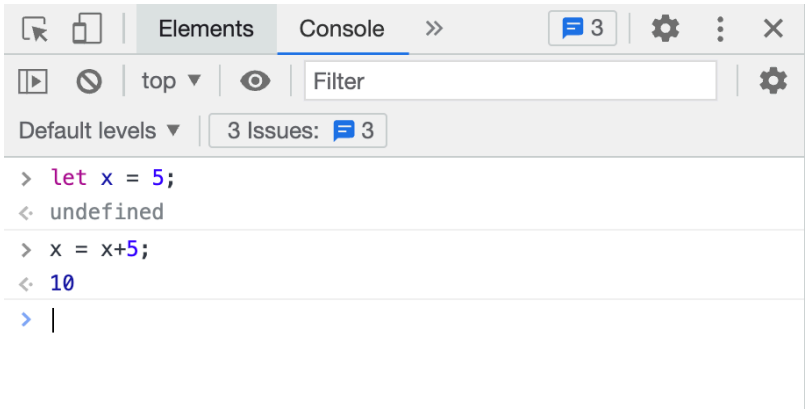
This is exactly equivalent to writing:

```
x = x + 1;
```



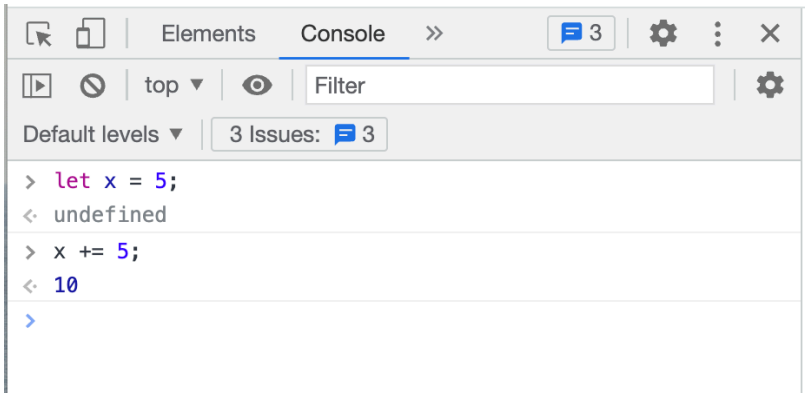
What if we want to add 10 to the existing value of x? Then the above won't work, but we can write:

```
x = x + 10;
```



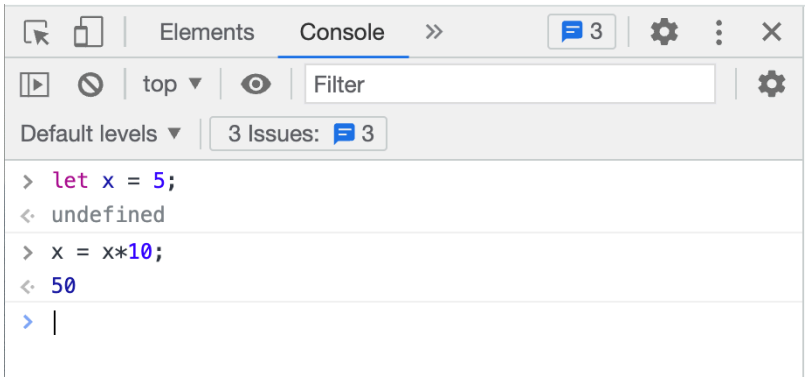
Or

```
x += 10;
```



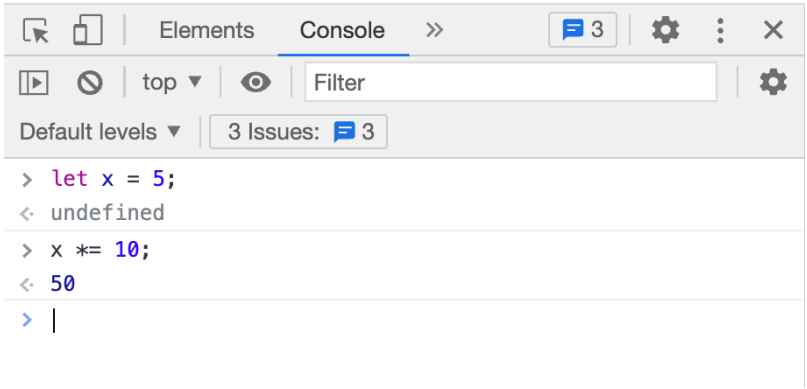
It works exactly the same way for the other math operators, too. For example, multiplying:

```
x = x*10;
```



Or

```
x *= 10;
```



Use whichever style you prefer.

8: What is a function or method?

A function is a chunk or block of code that

- takes in some inputs
- processes these inputs
- puts out some outputs

Think of a function like a little machine:

```
inputs --> |f| --> outputs
```

For example, the `typeof(variable)` you saw in the Lecture today took in the variable name as input, determined what type of data it was, and then put out as the output whether the variable was a string, number, or something else.

Another way of thinking about a function is through mathematics:

```
y = f(x)
```

Here, `y` is the output, `x` is the input. `x` is eaten in by the function `f`, processed in some way, and the result is `y`. A concrete example:

$$y = x^2$$

We have seen something like this before:

```
Area = f(Side Length)
```

In math, this is simply written as:

$$Area = side^2$$

So, the idea behind a function is that if you want to do something over and over again (like compute the area of a 100,000 squares for your generative artwork), you write it down as a block of code, and then name this block of code as the function name, (say `computeArea`) and then you can reuse it again and again by simply calling the function by its name and passing inputs (`side`) into it.

We will learn how to write lots and lots of functions (in fact writing functions is like bread and butter for programmers), but we will first learn how to use some functions that have been pre-written for us and are a part of JS. These **functions**, or **methods** as they are called, allow us to do many repetitive useful tasks. We are simply reusing functions that others have already written. In JS, a standard way of calling methods is to attach the function name with a dot (.) at the end of a variable name. For example:

```
document.getElementById("id")
```

This simply means that the method `getElementById()` goes to the `document` object, which is the root object of the HTML page, and fetches the Element object whose id matches with the input "`id`". While this is new, you should get used to this dot (.) notation, as we will see later that it is very common to perform method chaining in JS by simply attaching one method after another by either using the dot notation or by nesting methods.

That's all for today! Questions?